Black-box test design

Person Test Data Generator API · equivalence partitioning, boundary value analysis and decision tables

Item	Detail
System under test	FastAPI backend (Python, pydantic v2) that generates and validates Danish test persons
Test framework	pytest, parametrised tests with named partitions (pytest.param ids)
Techniques	Equivalence partitioning (EP), boundary value analysis (BVA), decision tables, error guessing
Scope	Input validation of PostalCode, Address and Person; the persons count parameter; properties of generated data
Latest recorded run	146 tests, all passed

1. Purpose and scope

This document explains how the test cases for the input validation of the API were derived with black-box design techniques. The test cases come from the specified rules of each input (type, length, range, format), not from the structure of the code. Every test case in the tables below is traceable to a partition or a boundary, and the last column gives the pytest id so the case can be found in the suite.

The database access layer (`postal_code_repository`) is outside the scope of this document. It is covered separately by an integration test against MariaDB and by structural (white-box) tests, which are not derived with the techniques described here.

2. Techniques and conventions

2.1 Equivalence partitioning

The set of possible values of each input is split into classes that the system must treat the same way. One representative per class is tested, so a class counts as covered by a single test. **Valid** classes (IDs containing `V`) must be accepted. **Invalid** classes (IDs containing `I`) must be rejected.

2.2 Boundary value analysis

Faults cluster at the edges of ranges and lengths. For each bounded input the value just below, at and just above each boundary is tested. Examples: a postal code of 3, 4 and 5 characters, a house number of 0, 1, 999 and 1000.

2.3 Decision tables

Where the outcome depends on a combination of conditions (the CPR number, the door field and the mobile phone number), a decision table lists every combination of conditions and the expected outcome (section 8).

2.4 Error guessing

Partitions that follow from experience with the type of defect rather than from the written rule: non-ASCII digits, values with trailing whitespace, wrong casing, empty strings, wrong data types and missing fields.

2.5 Single-fault rule

Every invalid test case violates exactly one rule and keeps all other fields valid. When the test fails or passes, it is clear which rule was responsible. The tests also check that the error is reported on the intended field (the error location must equal the field under test), so a rejection caused by a different field does not count.

2.6 Test structure

Each input has its own test function and one parametrised test per class group, for example valid and invalid partitions of the floor field. Shared valid objects come from fixtures in `tests/conftest.py`; a test overrides one field of a valid object, which keeps the single-fault rule.

3. Specified rules under test

The rules below are the specification from which the partitions are derived.

Input	Rule
`PostalCode.postal_code`	String of exactly 4 ASCII digits (leading zero allowed)
`PostalCode.town`	String; accepted as town or town_name
`Address.street`	String
`Address.number`	1 to 999 without leading zero, optionally followed by one uppercase letter A to Z
`Address.floor`	st, or a number from 1 to 99 without leading zero
`Address.door`	th, mf or tv; or a number from 1 to 50 without leading zero; or a lowercase letter, an optional dash and 1 to 3 digits
`Address.postal_code`	A valid PostalCode (object or dictionary)
`Person.first_name, last_name`	String
`Person.gender`	Exactly male or female (case sensitive)
`Person.date_of_birth`	A valid calendar date
`Person.cpr`	Exactly 10 ASCII digits; the first six must equal the date of birth as DDMMYY
`Person.mobile_phone_number`	Exactly 8 ASCII digits that start with one of the allowed Danish mobile prefixes (1, 2 or 3 digits long)
`Persons count (query parameter)`	Integer from 2 to 100

4. PostalCode

Tests: tests/test_models/test_postal_code.py

4.1 postal_code

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PC-V1	4 ASCII digits	2200	**Accept**	test_valid_postal_code_format[typical]
PC-V2	4 digits with leading zero	0800	**Accept**	test_valid_postal_code_format[leading-zero]
PC-I1	Empty string	(empty)	**Reject**: 4 digits	test_invalid_postal_code_format[empty]
PC-I2	Length 3 (boundary below)	123	**Reject**: 4 digits	test_invalid_postal_code_format[one-below-length]
PC-I3	Length 5 (boundary above)	22000	**Reject**: 4 digits	test_invalid_postal_code_format[one-above-length]
PC-I4	Letters only	abcd	**Reject**: 4 digits	test_invalid_postal_code_format[letters-only]
PC-I5	Letters mixed with digits	22a0	**Reject**: 4 digits	test_invalid_postal_code_format[letters-mixed-with-digits]
PC-I6	Four non-ASCII digits (Arabic-Indic)	Arabic-Indic digits for 2200	**Reject**: 4 digits	test_invalid_postal_code_format[non-ascii-digits]
PC-I7	Wrong type: integer	2200	**Reject**	test_wrong_type_is_rejected[postal_code-int]
PC-I8	Wrong type: None	None	**Reject**	test_wrong_type_is_rejected[postal_code-none]
PC-I9	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[postal_code]

4.2 town

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PT-V1	ASCII name	Aarhus C	**Accept**	test_valid_town[ascii]
PT-V2	Danish letters and hyphen	Høje-Taastrup	**Accept**	test_valid_town[danish-letters]
PT-V3	Supplied under the alias town_name	town_name=Aarhus C	**Accept**	test_town_accepts_town_name_alias
PT-I1	Wrong type: integer	123	**Reject**	test_wrong_type_is_rejected[town-int]
PT-I2	Wrong type: None	None	**Reject**	test_wrong_type_is_rejected[town-none]
PT-I3	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[town]

4.3 Whole object

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PO-V1	All fields valid (fixture)	2200, København N	**Accept**	test_valid_postal_code
PO-V2	Dump to dictionary and validate again	round trip	**Accept**: equal	test_model_dump_round_trip

5. Address

Tests: tests/test_models/test_address.py

5.1 street

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AS-V1	ASCII	Guldbergsgade	**Accept**	test_valid_street[ascii]
AS-V2	Danish letters	Østerbrogade	**Accept**	test_valid_street[danish-letters]
AS-V3	Punctuation and spaces	H.C. Andersens Boulevard	**Accept**	test_valid_street[punctuation-and-spaces]
AS-I1	Wrong type: integer	123	**Reject**	test_wrong_type_is_rejected[street-int]
AS-I2	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[street]

5.2 number (1 to 999, optional uppercase suffix)

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AN-V1	Lower boundary	1	**Accept**	test_valid_number[lower-boundary]
AN-V2	Typical value	29	**Accept**	test_valid_number[typical]
AN-V3	Upper boundary	999	**Accept**	test_valid_number[upper-boundary]
AN-V4	Number with suffix	29A	**Accept**	test_valid_number[with-suffix]
AN-V5	Upper boundary with last suffix	999Z	**Accept**	test_valid_number[max-with-suffix]
AN-I1	Empty string	(empty)	**Reject**: number rule	test_invalid_number[empty]
AN-I2	Suffix without number	A	**Reject**: number rule	test_invalid_number[letter-only]
AN-I3	Zero (boundary below)	0	**Reject**: number rule	test_invalid_number[zero]
AN-I4	Leading zero	01	**Reject**: number rule	test_invalid_number[leading-zero]
AN-I5	1000 (boundary above)	1000	**Reject**: number rule	test_invalid_number[above-upper-boundary]
AN-I6	Lowercase suffix	29a	**Reject**: number rule	test_invalid_number[lowercase-suffix]
AN-I7	Two-letter suffix	29AB	**Reject**: number rule	test_invalid_number[two-letter-suffix]
AN-I8	Letter inside the number	2A9	**Reject**: number rule	test_invalid_number[letter-in-the-middle]
AN-I9	Trailing space	29	**Reject**: number rule	test_invalid_number[trailing-space]
AN-I10	Non-ASCII digit	Arabic-Indic digit 3	**Reject**: number rule	test_invalid_number[non-ascii-digit]
AN-I11	Wrong type: integer	29	**Reject**	test_wrong_type_is_rejected[number-int]

5.3 floor (st or 1 to 99)

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AF-V1	Ground floor	st	**Accept**	test_valid_floor[ground-floor]
AF-V2	Lower boundary	1	**Accept**	test_valid_floor[lower-boundary]
AF-V3	Upper boundary	99	**Accept**	test_valid_floor[upper-boundary]
AF-I1	Empty string	(empty)	**Reject**: floor rule	test_invalid_floor[empty]
AF-I2	Zero (boundary below)	0	**Reject**: floor rule	test_invalid_floor[zero]
AF-I3	Leading zero	01	**Reject**: floor rule	test_invalid_floor[leading-zero]
AF-I4	100 (boundary above)	100	**Reject**: floor rule	test_invalid_floor[above-upper-boundary]
AF-I5	Ground floor, wrong case	ST	**Reject**: floor rule	test_invalid_floor[ground-floor-wrong-case]
AF-I6	Digit followed by letter	1a	**Reject**: floor rule	test_invalid_floor[digit-and-letter]
AF-I7	Negative number	-1	**Reject**: floor rule	test_invalid_floor[negative]
AF-I8	Non-ASCII digit	Arabic-Indic digit 3	**Reject**: floor rule	test_invalid_floor[non-ascii-digit]
AF-I9	Wrong type: integer	1	**Reject**	test_wrong_type_is_rejected[floor-int]

5.4 door

The door field has three valid forms, so the partitions are grouped by form. The decision table in section 8.2 shows how the forms relate.

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AD-V1	Special word th	th	**Accept**	test_valid_door[th]
AD-V2	Special word mf	mf	**Accept**	test_valid_door[mf]
AD-V3	Special word tv	tv	**Accept**	test_valid_door[tv]
AD-V4	Number, lower boundary	1	**Accept**	test_valid_door[number-lower-boundary]
AD-V5	Number, upper boundary	50	**Accept**	test_valid_door[number-upper-boundary]
AD-V6	Letter and 1 digit (lower boundary)	c3	**Accept**	test_valid_door[letter-one-digit]
AD-V7	Letter, dash and 2 digits	d-14	**Accept**	test_valid_door[letter-dash-two-digits]
AD-V8	Letter and 3 digits (upper boundary)	a123	**Accept**	test_valid_door[letter-three-digits]
AD-V9	Letter, dash and 3 digits	b-999	**Accept**	test_valid_door[letter-dash-three-digits]
AD-I1	Empty string	(empty)	**Reject**: door rule	test_invalid_door[empty]
AD-I2	Number 0 (boundary below)	0	**Reject**: door rule	test_invalid_door[zero]
AD-I3	Number with leading zero	01	**Reject**: door rule	test_invalid_door[leading-zero]

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AD-I4	Number 51 (boundary above)	51	**Reject**: door rule	test_invalid_door[above-number-boundary]
AD-I5	Special word, wrong case	TH	**Reject**: door rule	test_invalid_door[special-word-wrong-case]
AD-I6	Letter without digits	c	**Reject**: door rule	test_invalid_door[letter-only]
AD-I7	Letter and dash, no digits	c-	**Reject**: door rule	test_invalid_door[letter-dash-no-digits]
AD-I8	Uppercase letter	C3	**Reject**: door rule	test_invalid_door[uppercase-letter]
AD-I9	Letter and 4 digits (boundary above)	c1234	**Reject**: door rule	test_invalid_door[four-digits]
AD-I10	Double dash	c--3	**Reject**: door rule	test_invalid_door[double-dash]
AD-I11	Digit before letter	3c	**Reject**: door rule	test_invalid_door[digit-before-letter]
AD-I12	Trailing letter	c3x	**Reject**: door rule	test_invalid_door[trailing-letter]
AD-I13	Non-ASCII digit as number	Arabic-Indic digit 3	**Reject**: door rule	test_invalid_door[non-ascii-digit-number]
AD-I14	Non-ASCII digit after letter	c + Arabic-Indic digit 3	**Reject**: door rule	test_invalid_door[non-ascii-digit-suffix]
AD-I15	Wrong type: integer	5	**Reject**	test_wrong_type_is_rejected[door-int]

5.5 postal_code (nested object)

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
AP-V1	PostalCode object (fixture)	2200, København N	**Accept**	test_valid_address
AP-V2	Dictionary with town_name key	8000, Aarhus C	**Accept**	test_postal_code_accepted_as_dict
AP-I1	Wrong type: string	2200	**Reject**	test_postal_code_wrong_type_is_rejected[string]
AP-I2	Wrong type: None	None	**Reject**	test_postal_code_wrong_type_is_rejected[none]
AP-I3	Nested postal code invalid (3 digits)	123	**Reject**: at postal_code.postal_code	test_nested_invalid_postal_code_is_rejected
AP-I4	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[postal_code]

5.6 Whole object

Missing-field tests exist for street, number, floor and door (test_missing_required_field_is_rejected[...]). The serialisation rule, where the town is output as town_name, is verified by `test_postal_code_is_serialized_with_town_name`, and the round trip by `test_model_dump_round_trip`.

6. Person

Tests: tests/test_models/test_person.py

6.1 first_name and last_name

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PN-V1	ASCII name	John	**Accept**	test_valid_names[ascii]
PN-V2	Danish letters	Søren	**Accept**	test_valid_names[danish-letters]
PN-V3	Double-barrelled name	Anne-Marie	**Accept**	test_valid_names[double-barrelled]
PN-I1	Wrong type: integer (first_name)	123	**Reject**	test_wrong_type_is_rejected[first_name-int]
PN-I2	Wrong type: None (last_name)	None	**Reject**	test_wrong_type_is_rejected[last_name-none]
PN-I3	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[first_name]

6.2 gender

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PG-V1	male	male	**Accept**	test_valid_gender[male]
PG-V2	female	female	**Accept**	test_valid_gender[female]
PG-I1	Wrong case	Male	**Reject**	test_invalid_gender[wrong-case]
PG-I2	Value not in the set	other	**Reject**	test_invalid_gender[unlisted-value]
PG-I3	Empty string	(empty)	**Reject**	test_invalid_gender[empty]
PG-I4	Trailing whitespace	male	**Reject**	test_invalid_gender[trailing-whitespace]
PG-I5	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[gender]

6.3 date_of_birth

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PD-V1	Regular date	1990-01-17	**Accept**	test_valid_person (fixture)
PD-V2	Leap day	2000-02-29	**Accept**	test_cpr_matches_date_of_birth[leap-day]
PD-V3	First day of year	2005-01-01	**Accept**	test_cpr_matches_date_of_birth[leading-zeros]
PD-V4	Last day of year	1999-12-31	**Accept**	test_cpr_matches_date_of_birth[last-day-of-year]
PD-I1	Impossible calendar date	1990-02-30	**Reject**	test_invalid_date_of_birth[impossible-date]
PD-I2	Text that is not a date	not-a-date	**Reject**	test_invalid_date_of_birth[unparseable-string]
PD-I3	Wrong type: None	None	**Reject**	test_wrong_type_is_rejected[date_of_birth-none]
PD-I4	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[date_of_birth]

6.4 cpr, format

A valid CPR number must also match the date of birth, so the valid format partitions are exercised together with the matching date in section 6.5. The invalid format partitions keep the date valid and change only the CPR.

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
CF-I1	Empty string	(empty)	**Reject**: 10 digits	test_invalid_cpr_format[empty]
CF-I2	Length 9 (boundary below)	170190123	**Reject**: 10 digits	test_invalid_cpr_format[one-below-length]
CF-I3	Length 11 (boundary above)	17019012345	**Reject**: 10 digits	test_invalid_cpr_format[one-above-length]
CF-I4	Letters only	abcdefghij	**Reject**: 10 digits	test_invalid_cpr_format[letters-only]
CF-I5	Letters mixed with digits	12345abcde	**Reject**: 10 digits	test_invalid_cpr_format[letters-mixed-with-digits]
CF-I6	Non-ASCII digit at the end	123456789²	**Reject**: 10 digits	test_invalid_cpr_format[non-ascii-digit]
CF-I7	Wrong type: integer	1701901234	**Reject**	test_wrong_type_is_rejected[cpr-int]
CF-I8	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[cpr]

6.5 cpr compared with date_of_birth

The first six digits of the CPR number are the date of birth as DDMMYY. Each invalid case differs from the fixture date (17 January 1990, 170190) in exactly one component.

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
CD-V1	Regular date	1990-01-17 / 1701901234	**Accept**	test_cpr_matches_date_of_birth[regular-date]
CD-V2	Leading zeros in the CPR	2005-01-01 / 0101051234	**Accept**	test_cpr_matches_date_of_birth[leading-zeros]
CD-V3	Leap day	2000-02-29 / 2902001234	**Accept**	test_cpr_matches_date_of_birth[leap-day]
CD-V4	Last day of year	1999-12-31 / 3112991234	**Accept**	test_cpr_matches_date_of_birth[last-day-of-year]
CD-I1	Day differs	1801901234	**Reject**: mismatch	test_cpr_date_of_birth_mismatch[day-differs]
CD-I2	Month differs	1702901234	**Reject**: mismatch	test_cpr_date_of_birth_mismatch[month-differs]
CD-I3	Year differs	1701911234	**Reject**: mismatch	test_cpr_date_of_birth_mismatch[year-differs]

6.6 mobile_phone_number

Length and digits are checked before the prefix, so the invalid format cases use a valid prefix (42) and the invalid prefix cases use a valid length. The prefix partitions follow the list of allowed prefixes: prefixes of one digit (2), two digits (for example 30) and three digits (for example 342), with the first and last prefixes of the list and near-misses.

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
MV-V1	One-digit prefix	21234567	**Accept**	test_valid_mobile_phone_number[one-digit-prefix]
MV-V2	Two-digit prefix	30123456	**Accept**	test_valid_mobile_phone_number[two-digit-prefix]
MV-V3	Three-digit prefix	34212345	**Accept**	test_valid_mobile_phone_number[three-digit-prefix]
MV-V4	Last prefix in the list	82912345	**Accept**	test_valid_mobile_phone_number[last-listed-prefix]
MF-I1	Empty string	(empty)	**Reject**: 8 digits	test_invalid_mobile_phone_format[empty]
MF-I2	Length 7 (boundary below)	4210203	**Reject**: 8 digits	test_invalid_mobile_phone_format[one-below-length]
MF-I3	Length 9 (boundary above)	421020301	**Reject**: 8 digits	test_invalid_mobile_phone_format[one-above-length]
MF-I4	Letters	abcdefgh	**Reject**: 8 digits	test_invalid_mobile_phone_format[letters]
MF-I5	Non-ASCII digit	4210203²	**Reject**: 8 digits	test_invalid_mobile_phone_format[non-ascii-digit]
MP-I1	First digit not allowed	12345678	**Reject**: invalid prefix	test_invalid_mobile_phone_prefix[unlisted-first-digit]
MP-I2	Near-miss of a 3-digit prefix	34012345	**Reject**: invalid prefix	test_invalid_mobile_phone_prefix[near-miss-three-digit-prefix]
MP-I3	Near-miss of a 2-digit prefix	80123456	**Reject**: invalid prefix	test_invalid_mobile_phone_prefix[near-miss-two-digit-prefix]
MP-I4	Wrong type: integer	42102030	**Reject**	test_wrong_type_is_rejected[mobile-int]
MP-I5	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[mobile_phone_number]

6.7 address and whole object

ID	Partition / class	Representative value(s)	Expected result	Test case (pytest id)
PA-V1	Valid Address object	fixture	**Accept**	test_valid_person
PA-I1	Wrong type: string	not-an-address	**Reject**	test_wrong_type_is_rejected[address-string]
PA-I2	Field missing	(field omitted)	**Reject**	test_missing_required_field_is_rejected[address]

7. Persons count and generated data

7.1 Persons count parameter

Specified range: integer from 2 to 100. Test: tests/test_routes/test_person_route.py::test_persons_count_bounds

Class	Designed boundary / representative values	Expected
Below range (invalid)	`1`, `0`, `-1`	Reject (HTTP 422)
Lower boundary (valid)	`2`, `3`	Accept, that many persons returned
Inside range (valid)	`50`	Accept
Upper boundary (valid)	`99`, `100`	Accept, that many persons returned
Above range (invalid)	`101`	Reject (HTTP 422)
Wrong type (invalid)	`abc`	Reject (HTTP 422)

7.2 Generated data

The generators have no validated input to partition. They are tested black-box on their output, using the same rules as the models as the oracle.

Property of the output	Test
The same seed gives the same output (determinism)	`test_seeded_generators_are_repeatable`
The CPR number contains the date of birth, and its last digit has the correct parity for the gender	`test_cpr_contains_date_and_gender_parity`
The phone number has 8 digits and an allowed prefix	`test_phone_number_has_valid_length_and_prefix`
A generated address passes the Address model validation	`test_generated_address_passes_model_validation`
A generated person passes the Person model validation	`test_generated_person_passes_model_validation`
The API response has the expected keys and types	`test_api_contract.py::test_api_response_keys_and_types`

Note: the Person model validates that the CPR number matches the date of birth, but not the parity of the last digit against the gender. Parity is therefore checked on the generator output only.

8. Decision tables

A decision table covers combinations of conditions. In each table, T means the condition holds, F that it does not, and a dash that the condition is not evaluated or does not matter.

8.1 CPR number

Conditions are evaluated in this order. The comparison with the date of birth only runs when the field itself is valid, because the cross-field rule runs after field validation.

Condition / outcome	R1	R2	R3	R4
C1 Length is 10	T	F	T	T
C2 All characters are ASCII digits	T	-	F	T
C3 First six digits equal DDMMYY of date_of_birth	T	-	-	F
Outcome	Accept	Reject: 10 digits	Reject: 10 digits	Reject: date mismatch
Test cases	CD-V1 to CD-V4	CF-I1, CF-I2, CF-I3	CF-I4, CF-I5, CF-I6	CD-I1, CD-I2, CD-I3

8.2 Door

A: value is th, mf or tv. B: all characters are ASCII digits. C: number from 1 to 50 without leading zero. D: lowercase letter, optional single dash, then 1 to 3 ASCII digits.

Condition / outcome	R1	R2	R3	R4	R5
A Special word	T	F	F	F	F
B Digits only	-	T	T	F	F
C Number 1 to 50, no leading zero	-	T	F	-	-
D Letter, optional dash, 1 to 3 digits	-	-	-	T	F
Outcome	Accept	Accept	Reject	Accept	Reject
Test cases	AD-V1 to V3	AD-V4, AD-V5	AD-I2, I3, I4	AD-V6 to V9	AD-I1, I5 to I14

8.3 Mobile phone number

E1: length is 8. E2: all characters are ASCII digits. E3: the number starts with an allowed prefix.

Condition / outcome	R1	R2	R3	R4
E1 Length is 8	T	F	T	T
E2 All characters are ASCII digits	T	-	F	T
E3 Starts with an allowed prefix	T	-	-	F
Outcome	Accept	Reject: 8 digits	Reject: 8 digits	Reject: invalid prefix
Test cases	MV-V1 to MV-V4	MF-I1, MF-I2, MF-I3	MF-I4, MF-I5	MP-I1, MP-I2, MP-I3

9. Boundary value summary

Below, at and above each boundary. Values marked invalid are rejected, values marked valid are accepted.

Input	Boundary	Below (invalid)	At (valid)	Above (invalid)
postal_code length	4 characters	`123`	`2200`	`22000`
Address.number	1 to 999	`0`	`1`, `999`	`1000`
Address.floor	1 to 99	`0`	`1`, `99`	`100`
Address.door, number	1 to 50	`0`	`1`, `50`	`51`
Address.door, digits after letter	1 to 3 digits	`c`, `c-`	`c3`, `a123`	`c1234`
cpr length	10 characters	`170190123`	`1701901234`	`17019012345`
mobile_phone_number length	8 characters	`4210203`	`42102030`	`421020301`
Persons count	2 to 100	`1`	`2`, `100`	`101`

10. Results and findings

10.1 Results

Test file	Tests	Result
tests/test_models/test_address.py	68	All passed
tests/test_models/test_person.py	50	All passed
tests/test_models/test_postal_code.py	19	All passed
tests/test_generators/test_generators.py	3	All passed
tests/test_services/test_generation_services.py	2	All passed
tests/test_database/test_database.py	2	All passed
tests/test_routes/test_person_route.py	1	All passed
tests/test_api_contract.py	1	All passed
Total	146	All passed

The model modules (address, person, postal_code, person_responses and validators) reach 100% line and branch coverage. Coverage is only a secondary check here: the cases were chosen from the rules, and coverage confirms that no branch of the validation code is left untested.

10.2 What the design revealed

- **Non-ASCII digits were accepted.** Python's `str.isdigit()` returns true for digits from other scripts, so values such as Arabic-Indic digits passed the postal code, number, floor and door rules. The error-guessing partitions (PC-I6, AN-I10, AF-I8, AD-I13, AD-I14) exposed this, and all digit checks now use one shared helper (`is_ascii_digits` in `app/models/validators.py`).
- **Original invalid-address test combined four faults.** A single case with an invalid street, number, floor and door passes as soon as any one rule rejects it. It was replaced by one case per partition, which follows the single-fault rule.
- **No rule for empty names.** The specification does not constrain first_name, last_name, street or town beyond being strings, so an empty string is accepted and no test asserts either way. This is an open decision: if empty values must be rejected, add a minimum length to the model and an empty-string partition for each field.